

these controls or configure them loosely. This enables the attacker to break into the system very easily.

How to fix it: Harden the system, i.e., disable unnecessary functionality in the underlying applications, patch and update the systems regularly, and keep track of the libraries used in the Web application (update, if the new version is available or discard if it's no longer supported). Regularly run vulnerability scans and penetration tests to uncover flaws.

6. Exposure to sensitive data

One of the most common flaws in Web applications is their failure to protect sensitive data adequately. They either store or transmit data in plain text, use weak encryption and hashing algorithms, or the keys aren't properly managed. Exploitation of all or any one of these can have a massive negative impact on a business' reputation.

How to fix it: Use FIPS-certified data encryption algorithms, collect and store only what's necessary, delete the data once it has served its purpose and disable auto-complete.

7. Missing function-level access control

This flaw occurs when the access to application functions isn't validated. For example, a Web application may hide the link to the *Administration* console by not presenting it to a normal user. However, it does not implement any function level access control to check who is accessing administration functions. As a result, normal users can access administration functions, provided they know which ones to use.

How to fix it: Implement access control on all functions handling sensitive data or functionality. At the least, assign roles to the users and validate their roles before granting access to sensitive functions.

8. Cross-Site Request Forgery (CSRF)

CSRF occurs when an attacker can craft a Web page that requests the user's browser to send session information of some other website (e.g., *sample.com*). The browser senses this as a valid request and sends the requested information to the attacker's server. Thus the attacker can forge the user's session on *sample.com* with the stolen information.

How to fix it: Hide CSRF tokens in the body of the Web page, use CAPTCHA as a means for users to confirm their actions, etc.

9. Using components with known vulnerabilities

Most Web applications use third party libraries to implement certain functionality. While this saves on development efforts and time, it may become a developer's nightmare if such libraries are not managed properly. For example, consider a Web application using a certain reporting library, which has reached its end-of-support phase. If the developers fail to keep track of this library, their application will be vulnerable to future vulnerabilities affecting the library.

How to fix it: Keep track of the versions, patches and upgrades of the libraries in use; discard libraries that are no longer in development or have reached the end-of-support

phase. Create a policy outlining the process for safe usage of such libraries.

10. Invalid redirects and forwards

Redirects and forwards are a common occurrence in a Web application and it's important to check whether or not they lead to a valid or correct page. If attackers can modify the parameters of the redirection URL and change the landing page, they can bypass the website's security mechanisms or lead users to disclose sensitive information.

How to fix it: Avoid using redirects and forwards. If they can't be avoided, ensure that no user parameters are involved in generating the landing page. If user parameters are to be used, ensure that they are properly validated.

Additional measures to secure Web applications

OWASP's ranking of the top ten vulnerabilities enables developers to secure their Web applications against the most prevalent vulnerabilities but this complex task doesn't end here. Once these are taken care of, Web developers should look at the environments in which their application will be used, who are its target users, etc. For example, if the application is being developed for military purposes, it'll need to have strong access control mechanisms. If it is to be used in the health care industry, then encryption and the protection of data from leakage takes priority.

The IT environment differs from one organisation to another and each environment has its set of vulnerabilities. Web applications developers must take this factor into consideration and secure their applications with respect to the environment in which the Web application will operate.

Protecting applications from hobby hackers is a thing of the past. Attackers now have advanced skill sets and use sophisticated techniques to crack websites. Day in and day out there's news of some website being hacked and sensitive data being leaked (e.g., Dropbox, Apple iCloud, RSA, Snapchat, etc). Cracking into websites has become a booming business in the online underworld.

While attackers only have to find a new attack vector, Web developers are tasked with protecting their applications against known attacks as well as those that are yet unknown. The OWASP community has done a lot to ease the burden of Web developers by publishing the list of the top 10 vulnerabilities. Based on this, developers can prioritise their development efforts and develop secure Web applications without compromising a project's deadline or the overall user experience. 

By: Uday Mittal

The author is an information security consultant, an open source enthusiast and likes to experiment with new technologies. He can be reached at mailme@udaymittal.com